

# 序列操作1

给定一个长度为 $n$ 的数组，再给定三个操作：

- 1  $x$ ：在数组末尾插入一个数 $x$
- 2：删除数组末尾的数
- 3  $k$ ：询问数组中前 $k$ 个数的最大值,  $k < \text{len}$ 。

输入：

3

5 2 7

2

1 6

3 2

输出：

5

操作数 $m \leq 1000000$ 。

# 解题思路

因为只是在末尾删除，末尾插入，所以很容易想到单调栈

查询需要查询前缀最大值，那么我们对于栈中的每一个位置，用一个数组 $\text{Max}[i]$ 表示栈中前 $i$ 个数的大值。

# 序列操作2

给定一个数组，再给定三个操作：

1 x: 在末尾插入数x并询问数x之前比x大的第一个数，如果没有，输出0。

初始是数组为空。

输入：

5

1 2

1 7

1 4

1 5

1 3

输出：

7

5

操作数 $m \leq 1000000$ .

# 序列操作2

- 1、和上题一样，只涉及在末尾插入，所以考虑栈来维护
- 2、需要查询离 $x$ 最近的一个最大值。



第 $x$   
个数

第 $y$   
个数

第 $z$   
个数

已知  $x < y < z$ ,  $h[x] < h[y]$   
求 $z$ 之前第一个大于 $z$ 的数  
如果 $h[z] < h[x]$  不选 $x$   
如果 $h[y] > h[z] > h[x]$  不选 $x$   
如果 $h[z] > h[y]$  不选 $x$



第 $x$   
个数

第 $y$   
个数

第 $z$   
个数

已知  $x < y < z$ ,  $h[x] > h[y]$   
求 $z$ 之前第一个大于 $z$ 的数  
如果 $h[y] < h[z] < h[x]$  选 $x$   
如果 $h[y] < h[z]$  选 $y$   
如果 $h[z] > h[x]$  不选



# 序列操作 2

始终记住单调栈中维护的是有用的信息，即后面可能会用到的信息，后面无论如何都不会被使用的信息就舍弃了。

既然是单调栈，那么说明就是单调的，而我们需要的最大最小值，要么在栈顶，要么在栈底。





# 序列操作2

始终记住单调栈中维护的是有用的信息，即后面可能会用到的信息，后面无论如何都不会被使用的信息就舍弃了。

既然是单调栈，那么说明就是单调的，而我们需要最大值，要么在栈顶，要么在栈底。



# 思考题

给定一个长度为 $n$ 的数列，再给定 $m$ 个操作，现在要求维护五个操作。

- 1: 1 X: 在光标的前面插入一个数字 $X$ 。
  - 2: 2: 删除光标前的最后一个数字，如果光标在第一个数字前，则忽略。
  - 3: 3: 左移光标，如果无法左移，则忽略。
  - 4: 4: 右移光标，如果无法右移，则忽略。
  - 5: 5 k: 求前 $k$ 个数的前缀和，保证 $k \leq$  光标当前所在位置。
- 初始时光标在最后一个数的末尾  
 $Q \leq 100000$ 。

# 序列操作3

给定一个数组，再给定三个操作：

- 1 x: 在数组末尾插入一个数x
  - 2: 删除数组中的第一个数
  - 3: 询问当前数组中前k个数之和
- 初始是数组为空。

输入：

1 2

1 7

1 4

2

3

输出：

11

操作数 $m \leq 1000000$ .



# 序列操作3

第一种暴力做法，每删除一个数，后面的数左移，然后求数组之和1—1，但是不但麻烦，效率还低。

我们完全可以开两个变量记录一下当前数组的有效区间，区间 $[l, r]$ 即表示当前数组的有效长度，那么区间之和如何快速求呢？完全没必要一个一个加，直接用前缀和相减就可以了。

# 序列操作4

给定一个数组，再给定三个操作：

- 1 x: 在数组末尾插入一个数x
- 2: 删除数组中的第一个数
- 3: 删除数组中的最后一个数
- 4: 查询数组中前k个数之和， $k \leq \text{len}$

初始时数组为空。

输入：

1 2

1 7

1 4

2

3

4

输出：

7

操作数 $m \leq 1000000$ .

# 序列操作 5

给定一个数组，再给定三个操作：

- 1 x: 在数组末尾插入一个数x
- 2: 删除数组中的第一个数
- 3: 查询当前数组中的最大值

初始时数组为空。

给定一个长度为L的数组，每插入一个数x，就删除数组中第一个元素，问数组中的最大数

1 9 9

1 7 8

1 8

3

2

3

操作数 $m \leq 1000000$ .

# 序列操作 5

先想到，求一个前缀最大值，然后得到最后的有效区间，然后。。。就没有然后了。

我们以前计算的和，只具有可加可减性的，即两个区间之和等价于两个区间合并之后的和，同样，求一个小区间的和，用一个大区间的和-一个小区间的和也是正确的

但是，最大值，很明显不满足这个条件。比如该题每个位置前缀最大值分别是 9 9 9, 删除第一个数，最大值还是9，但是9已经不存在了。

和前面一道题一样，我们思考开一个数组来维护有用信息

# 序列操作 5

序列 9 7 8

首先，输入第一个数9(后面的数目前都还不知道)，肯定是有用的，存进来。



然后再输入7

虽然7比9小，只要有9在7肯定不是最大值，但是会有删除操作啊，万一9被删除了呢？7说不定就是最大值了，所以还不能删，保留。



# 序列操作 5

继续输入8，我们思考一下此时8后面还没有其他数，很有可能8前面的数都被删除了，那么8可能就是最大值，那么8肯定是要放进来的。也就是说一个新的数我们肯定是要放进数组的。

当8存在的时候，7永远不可能是最大值。为什么呢？第一种情况求最大值的时候，7还未被删除，包含7的区间肯定包含8，有没有7都一样。7已经被删除，那更没有用了。所以7要被删除，





# 序列操作 5

那么，如果数据再长一点呢？9 7 6 8 8. 求最大值应该如何求解呢？



# 序列操作 6

给定一个数组，再给定三个操作：

- 1 x: 在数组末尾插入一个数x
  - 2: 查询当前数组中后L个数的最大值(L是一个固定值)
- 初始时数组为空，初始时给定L。

输入：

8 3

1 6

1 2

1 4

2

1 3

2

1 5

2

输出：

6

4

5

操作数 $m \leq 1000000$ .



# 队列

我们去食堂打饭，去超市买东西，去银行取钱，都需要做一件事情——排队。比如在打饭的时候，先到的人先打饭，后到的人后打饭，新来的人总是排在队尾。每次出队的都是队首的人。如图



队列是限定在一端进行插入，另一端进行删除的特殊线性表。允许出队的一端被称之为**队头**，允许入队的一端被称之为**队尾**

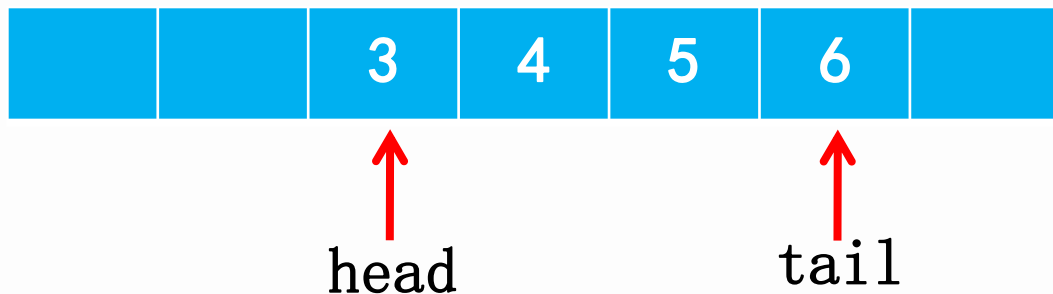
# 数组模拟队列

(1) 一个数组：用来存放队列中的元素



(2) 队头指针(head)：因为出队是在队头做操作，所以需要一个变量存储当前队头的位置

(3) 队尾指针(tail)：因为入队是在队尾做操作，所以需要一个变量存储当前队尾的位置



当队列为空时，怎么表示？

$\text{head} == \text{tail}$  ?



# 数组模拟队列

在平时使用过程中，为了方便，我们一般让head指针指向队头的位置，而tail指针指向队尾的下一个位置，这样我们就可以很容易判断出队列是否为空。 $head == tail$



↑  
head

↑  
tail

tail-head表示什么意思?



↑   ↑  
head tail

队列长度



↑   ↑  
head tail

# 队列操作

在队列中，只有两种操作：入队(push)和出队(pop)。入队是在队尾做操作，出队是在队头做操作。

head: 指向队头位置

tail: 指向队尾位置的下一个位置

入队(push):

`queue[tail]=x;`

`tail++;`

出队(pop):

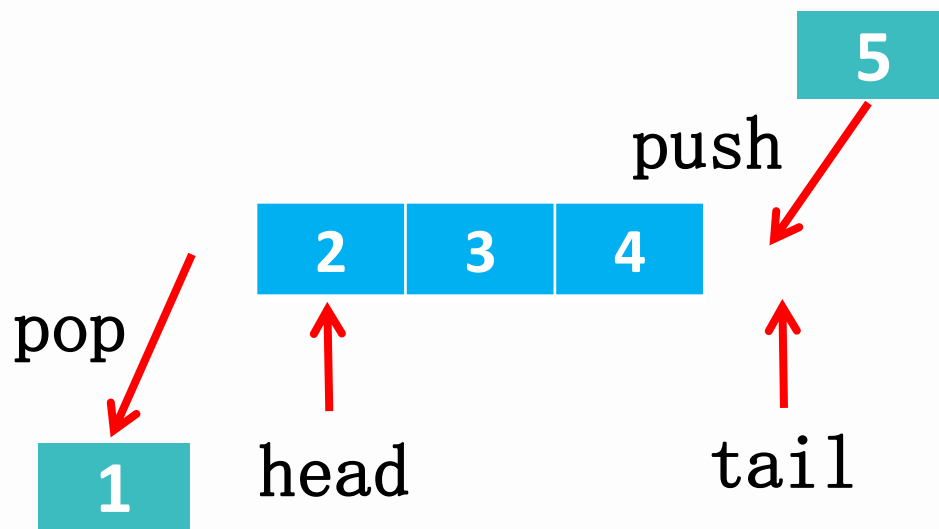
`head++;`

队列为空(empty):

`head==tail;`

队列长度(size):

`tail-head;`



# STL 队列

头文件: `#include<queue>`  
队列的定义: `queue<int> q;`  
入队(push): `q.push(x);`  
出队(pop): `q.pop();`  
队列为空: `q.empty();`  
队列的长度: `q.size();`  
输出队头元素: `x=q.front(); //不出队`  
输出队尾元素: `x=q.back(); //不入队`

最后两种操作仅适用于FIFO队列，即一般的队列。

注意：STL库里面的栈和队列都是严格按照该结构实现的，所以都不可以和普通数组一样遍历或者做其他操作。

# 广度优先搜索

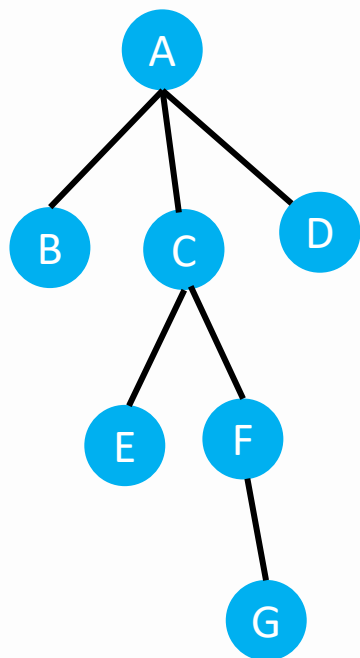
我们以前学习过一种搜索基本的搜索方法——广度优先搜索 (DFS), 用到了递归这种结构, 其实也就是栈的一个应用。我们今天将学习另外一种高效的搜索方法——广度优先搜索 (BFS)

如果按照深度优先搜索的方法遍历: A-B-C-E-F-G-D

如果按照广度优先搜索的方法遍历:

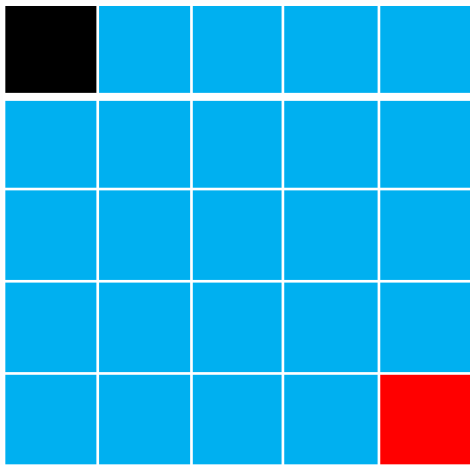
A-B-C-D-E-F-G

广搜就是指先找出当前位置的所有可能的位置, 再继续找出下一个位置的所有可能的位置



# 最短路径

在一个 $n \times n$ 的迷宫中，已知起点(黑色)和终点(红色)，迷宫中某些位置不可以走，求起点到终点的最短路径是多少。



|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 | 5 |
| 2 | 3 | 4 | 5 | 6 |
| 3 | 4 | 5 | 6 | 7 |
| 4 | 5 | 6 | 7 | 8 |

广度优先搜索：从初始结点开始，先找到第一层（走一步）的所有的结点。再遍历所有第一层的结点，找到左右第二层（走两步）的结点。这样一次找下去。当找到终点的时候，就是从起点到达终点的最短路径。

# 最短路径

|   |   |   |  |  |
|---|---|---|--|--|
| 0 | 1 | 2 |  |  |
| 1 | 2 |   |  |  |
| 2 |   |   |  |  |
|   |   |   |  |  |
|   |   |   |  |  |

|    |   |   |   |   |   |   |
|----|---|---|---|---|---|---|
| 行  | 1 | 1 | 2 | 1 | 2 | 3 |
| 列  | 1 | 2 | 1 | 3 | 2 | 1 |
| 步数 | 0 | 1 | 1 | 2 | 2 | 2 |

广度优先搜索其实就是队列的一个应用，首先，起点入队，找到起点能一步到达的所有合理的并且未标记的点，并且依次入队。然后起点出队，继续寻找队头的点的所有下一个点，找到之后入队。然后队头出队。

当队列为空的时候(找不到路径)或者找到终点的时候搜索结束



# 最短路径

在一个 $n*n$ 的迷宫中，0表示可以走，1表示障碍物，已知起点和终点，每次只能向上下左右的方向移动一步。求从起点到终点的最短步数

输入：

3

0 0 0

0 0 1

1 0 0

1 1 3 3

输出：

4

$n \leq 1000;$

注意：深度优先搜索由于会走很多回头路和重复的路，再加上递归的时间效率远远低于循环的时间效率。所以搜索的题。深搜不能搜索超过 $100*100$ 的迷宫。

# 解题思路

- (1) 把起点放入队列，此时队列里面只有起点一个元素。
- (2) 遍历起点的所有方向，找出所有合理的并且没有被标记过的点放入队列中。
- (3) 当队头元素所有方向遍历完成后，队头元素出队，继续遍历当前队头元素的所有方向并且依次入队。
- (4) 重复执行上述操作，直到队列为空或者找到终点为止。如果队列为空，说明从起点到终点没有路径，如果找到终点，则当前路径长度就是最短路径长度

```
struct Queue  
{  
    int x;  
    int y;  
    int step;  
};  
Queue q[1001000];
```

因为要记录当前位置的行列坐标和最短路径，所以需要用一个结构体

由于该题范围是 $n \leq 1000$ . 所以迷宫中最多 $1000 \times 1000$ 个点入队，所以队列必须 $> 1000^2$

# 最短路径

(1) 起点入队:

```
if (x==X&&y==Y) //特判起点和终点相同
{
    printf("0");
    return 0;
}
int head=1; //定义队头指针
int tail=1; //定义队尾指针指向队尾的下一个位置
q[tail].x=x; //起点入队
q[tail].y=y;
q[tail].step=0;
tail++; //队尾指针后移
book[x][y]=1; //标记起点
```

# 最短路径

```
while(head<tail) //当队列不为空的时候表示还可以继续搜索
{
```

```
    for(int k=0;k<=3;k++) //分别遍历上下左右四个方向
    {
```

```
        int tx=q[head].x+next[k][0];
        int ty=q[head].y+next[k][1];
        if(tx<1||ty<1||tx>n||ty>n) continue;
        if(map[tx][ty]==0&&book[tx][ty]==0)
        {
```

```
            if(tx==X&&ty==Y)
            {
```

```
                printf("%d", q[head].step+1);
                return 0;
            }
```

```
            book[tx][ty]=1;
```

```
            q[tail].x=tx;    q[tail].y=ty;
```

```
            q[tail].step=q[head].step+1;
```

```
            tail++; //入队之后队尾指针+1
```

如果找到了终点，因为该位置是当前队头位置移动一步到达的，所以用q[head].step+1表示

如果当队列为空时，表示所有可能都搜索完了，还没有找到终点，输出“NO”

```
head++; //当前队头元素所可能遍历完之后，出队
```

```
}
```

# STL 队列

定义一个队列:

```
#include<queue>
struct node
{
    int x;
    int y;
    int step;
};
queue<node> q;
```

起点入队:

```
node t; //定义一个结点
t.x=x;
t.y=y;
t.step=0;
q.push(t); //整个结点入栈
```

方法2: 构造函数:

```
struct node
{
    int x;
    int y;
    int step;
    node(x1, y1, step1)
    {
        x=x1;
        y=y1;
        step=step1;
    }
};
queue<node> q;
q.push(node(x, y, 0));
```

不能和上面使用  
相同的变量名

# STL 队列

```
while(!q.empty())
{
    node tmp=q.front(); //存储队头元素，但是不出队
    for(int k=0;k<=3;k++)
    {
        int tx=tmp.x+next[k][0];
        int ty=tmp.y+next[k][1];
        if(tx<1||tx>n||ty<1||ty>n) continue;
        if(map[tx][ty]==0&&book[tx][ty]==0)
        {
            if(tx==X&&ty==Y)
            {
                printf("%d", tmp.step+1);      return 0;
            }
            q.push(node(tx, ty, tmp.step+1)); //入队
            book[tx][ty]=1;
        }
    }
    q.pop(); //队头元素出队
}
```

# DFS 和 BFS

既然，广度优先搜索的效率远高于深度优先搜索，那么我们为什么还要学习深搜呢？广搜能不能完全代替深搜呢？

我们以前学习的深搜的题包括以下几类：

## 一：非最短路 (只能用DFS)

- (1) 起点到终点的所有路径
- (2) 起点到终点的所有方案

BFS只能用来求权值相同的最短路问题

## 二：最短路

- (1) 每走一步花费的代价一样 (DFS和BFS)
- (2) 每走一步花费的代价不一样 (只能用DFS)

综上搜索，因为BFS是层层搜索，所以找到的起点到终点的距离一定是最短路径，并且搜索过一次就不会再搜索第二次了。同样的，当权值不同的时候，层次就会发生错误，第一次求出来的未必就是最短路。



# D F S 和 B F S

